I use Typst to generate PDFs of my blog posts and my CV

Emil Lindfors | February 14, 2026

Every post on this blog has a PDF button in the sidebar. Click it and you get a properly typeset document – serif body text, sans-serif headings, colored code blocks, rendered math. It looks like something you'd find on arXiv, not a browser print-to-PDF.

My CV lives as a `.typ` file in the same git repo. When I push, it compiles to a PDF that's served at `/cv.pdf`. No Google Docs. No Overleaf. No emailing `CV_final_FINAL_v3.pdf` to myself.

Both are powered by Typst, a typesetting system written in Rust. If you've used LaTeX and wished it didn't hate you, Typst is the answer.

Why not just print to PDF?

Browser print-to-PDF produces ugly output. Navigation elements, footers, and sidebars leak in. Line breaks happen in wrong places. Code blocks get cut off. Math renders at screen resolution. Fonts fall back to system defaults.

You can add `@media print` CSS to fix some of this, but you're fighting the browser's layout engine the whole way. Print CSS is a maintenance burden that produces mediocre results.

I wanted PDFs that look *designed* – consistent typography, proper margins, page numbers, and a layout that works on paper. That means a real typesetting engine.

Why Typst over LaTeX?

I used LaTeX throughout my PhD. It produces gorgeous output. It also:

- Takes 30+ seconds to compile a document
- Produces error messages like `! Undefined control sequence. l.47 \begin{itemize}` when you forget a package
- Requires a 4GB TeX Live installation
- Has a syntax that looks like it was designed to be read by compilers, not humans
- Makes you choose between dozens of document classes, packages, and incompatible options before you can write a single paragraph

Typst fixes all of this:

	LaTeX	Typst
Compile time	30+ seconds	< 1 second
Binary size	~4 GB (TeX Live)	~30 MB
Error messages	Cryptic	Readable, with line numbers
Syntax	`\textbf{bold}`	`*bold*`
Package management	Manual `.sty` files	Built-in registry
Language	TeX macros	Real scripting language

Typst's syntax is close enough to markdown that it feels natural. Bold is `*bold*`. Italics is `_italics_`. Code blocks are triple backticks. Headings are `= Heading`. You can write a document without reading a manual.

But when you need power, Typst has a real programming language. Functions, variables, loops, conditionals. Not TeX's macro expansion nightmare, but actual code:

```
#let accent = rgb("#2c3e50")

#let section(title) = {
  v(0.6em)
  text(weight: "bold", size: 11pt, fill: accent)[#upper(title)]
  v(-0.3em)
  line(length: 100%, stroke: 0.5pt + accent)
  v(0.3em)
}
```

That's readable. Try doing the same in LaTeX.

My CV in Typst

My CV is a single file: `cv.typ` at the repo root. Here's how it starts:

```
#set document(title: "Emil Lindfors - CV", author: "Emil Lindfors")
#set page(paper: "a4", margin: (x: 1.8cm, y: 1.5cm))
#set text(font: "Libertinus Serif", size: 10pt)
#set par(justify: true)
```

Four lines and you have a fully configured document. Page size, margins, font, paragraph settings. In LaTeX this would be a `\documentclass` with options, plus `\usepackage{geometry}`, `\usepackage{fontspec}`, and a prayer that they don't conflict.

I define small helper functions for repeated structures:

```
#let experience(company, role, dates, location, bullets) = {
  grid(
    columns: (1fr, auto),
    text(weight: "bold")[#company],
    text(style: "italic", fill: light-gray)[#dates]
  )
  text(style: "italic")[#role]
  + h(1em)
  + text(fill: light-gray, size: 9pt)[#location]
  v(0.2em)
  for bullet in bullets {
    [- #bullet]
  }
  v(0.4em)
}
```

Then each job entry is a function call:

```
#experience(
  "AquaCloud",
  "Senior Software Engineer",
  "January 2025 -- Present",
  "Norway",
  (
    [Architected aquaculture data platform integrating
     Fishtalk, Mercatus, and other industry systems],
    [Built data pipelines for major Norwegian salmon farmers],
    // ...
  )
)
```

The CV compiles in under a second. The output is a clean, professional PDF with proper grid alignment, consistent spacing, and a photo in the header. It lives in version control alongside my blog, and updating it means editing one text file and pushing.

Building it

The build script compiles it alongside everything else:

```
typst compile cv.typ static/cv.pdf
```

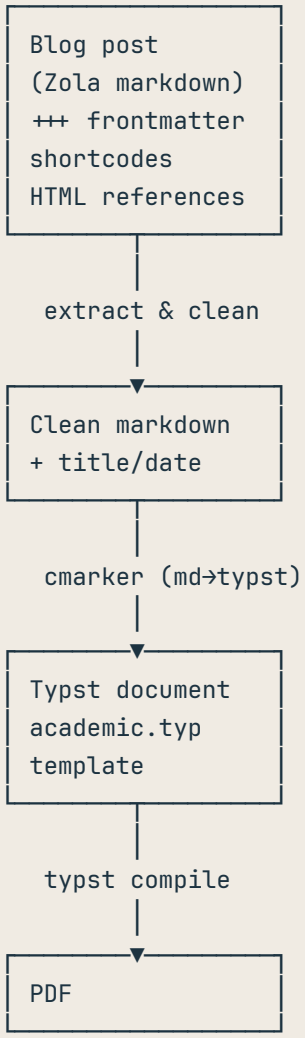
That's it. One command. The resulting PDF is served at `lindfors.no/cv.pdf`. Every page on the site links to it from the author sidebar.

Blog post PDFs

This is the more interesting part. Every blog post is written in markdown for Zola. I wanted a PDF for each one, but the content lives in Zola-flavored markdown with TOML frontmatter, shortcodes, and HTML citations. Typst doesn't understand any of that.

The solution is a three-stage pipeline: extract, preprocess, render.

The pipeline



Stage 1: Extract

A shell script splits the markdown file into frontmatter and body using `awk`:

```
awk '
BEGIN { in_front = 0; front_done = 0 }
/^\+\+\+$/ {
    if (in_front == 0) { in_front = 1; next }
    else { front_done = 1; in_front = 0; next }
```

```
}
in_front == 1 { print > "frontmatter.toml" }
front_done == 1 { print > "content.md" }
' "$INPUT_FILE"
```

Title and date are parsed from the TOML frontmatter with `grep` and `sed`. Not elegant, but it works and has zero dependencies.

Stage 2: Preprocess

The markdown body needs cleaning before Typst can handle it. Blog posts contain things that make sense on the web but not in a PDF:

Citation anchors like `1` become plain `1` – the PDF has the references right there on the page, no need for hyperlinks:

```
sed -i -E 's/\[([0-9]+)\]\(#ref-[^)]+\)/\1/g' content.md
```

HTML reference blocks get converted to markdown lists. On the web, references are `<p>` tags with classes for styling. In the PDF, they become simple bulleted items:

```
sed -i -E 's/<p[^>]*class="reference"[^>]*>/- /g' content.md
sed -i -E 's/<\/p>//g' content.md
sed -i -E 's/*/*/g' content.md
sed -i -E 's/<\/em>/*/g' content.md
sed -i -E 's/<a href="([^"]+)">([^<]+)<\/a>/[\2](\1)/g' content.md
```

After preprocessing, the markdown is clean enough for Typst to consume.

Stage 3: Render

The generated Typst document is minimal – it imports my template, sets the metadata, and renders the markdown:

```
#import "academic.typ": academic
#import "@preview/cmarker:0.1.8"
#import "@preview/mitex:0.2.6": mitex

#show: academic.with(
  title: "My Post Title",
  author: "Emil Lindfors",
  date: "February 10, 2026",
)

#cmarker.render(
  read("content.md"),
```

```
  math: mitex,
  smart-punctuation: true,
)
```

Two Typst packages do the heavy lifting:

- cmarker converts CommonMark markdown to Typst. It handles headings, lists, code blocks, links, images, tables – everything. The blog post markdown goes in, Typst content comes out.
- MiTeX renders LaTeX math syntax in Typst. The same `$E = mc^2$` that KaTeX renders on the web, MiTeX renders in the PDF. No syntax changes between formats.

The academic template

The `academic.typ` template controls the look of every blog post PDF. I designed it to match my website's visual identity:

```
#let academic(
  title: none,
  author: "Emil Lindfors",
  date: none,
  abstract: none,
  body
) = {
  // Sea theme colors (same as the website)
  let color-text = rgb("#1C3240")        // Deep Sea
  let color-accent = rgb("#2A8F82")      // Teal
  let color-link = rgb("#D4706A")        // Coral
  let color-bg-code = rgb("#F0EBE3")     // Tide Pool

  set page(paper: "a4", margin: (x: 2.5cm, y: 2.5cm))

  // Body text: Literata (serif) — same as the blog
  set text(
    font: ("Literata", "Libertinus Serif"),
    size: 11pt,
    fill: color-text,
  )

  // Headings: Inter (sans-serif) — same as the blog
  show heading.where(level: 1): it => {
    set text(
      font: ("Inter", "Ubuntu Sans"),
      size: 18pt, weight: 600,
    )
    v(1.5em)
    it.body
```

```
    v(0.75em)
  }

  // Code blocks: teal left border, light background
  show raw.where(block: true): it => {
    set text(font: "JetBrains Mono", size: 9pt)
    block(
      fill: color-bg-code,
      stroke: (left: 3pt + color-accent),
      inset: 10pt, radius: 4pt, width: 100%,
      it
    )
  }

  // ... title block, page footer, etc.
  body
}
```

The same fonts (Inter + Literata), the same colors (deep sea, teal, coral), the same code block styling (teal left border). Someone reading the blog and then opening the PDF sees the same visual language. The self-hosted fonts are passed to `typst compile` with `--font-path`:

```
typst compile \
  --font-path fonts/inter \
  --font-path fonts/literata \
  document.typ output.pdf
```

No system font dependencies. The PDF looks identical on any machine.

The build integration

The PDF generation steps in the deploy script are straightforward:

```
# Compile CV
typst compile cv.typ static/cv.pdf

# Generate blog post PDFs
for post in content/blog/*/index.md; do
    ./scripts/generate-pdf.sh "$post"
done
```

These run after citation processing (since the PDF generator needs resolved references) and before `zola build`. The generated PDFs go into `static/pdf/`, which Zola copies to the build output. Every post's sidebar has a PDF download button that links to `/pdf/{slug}.pdf`.

Typst compiles each post in well under a second. The whole site including all PDFs builds in a few seconds. Compare that to LaTeX, where a single document with `biblatex` can take 30 seconds and multiple passes. For the full build pipeline, see the site overview post.

What the output looks like

A blog post PDF has:

- **Title page header** with the post title in 22pt Inter, author and date centered below
- **A horizontal rule** separating the header from body content
- **Body text** in 11pt Literata with justified paragraphs
- **Headings** in Inter (18pt/14pt/12pt for h1/h2/h3)
- **Code blocks** with a teal left border on a light background, in JetBrains Mono
- **Blockquotes** with the same teal left border treatment
- **Math** rendered natively via MiTeX
- **Links** in coral, matching the website
- **Page numbers** centered in the footer (e.g., "3 / 7")

The CV has a different style – more compact, with a grid layout, photo in the header, and section dividers. Same Typst, different template.

Why this matters

Two reasons:

1. PDFs outlast websites. URLs break. Services shut down. My blog might not be here in 10 years. But a PDF saved to someone's hard drive will still open in 2040. Every blog post I write about aquaculture production costs or sensor architectures might be cited in someone's thesis. A PDF makes that possible.

2. The CV is never stale. My CV is a text file in the same repo as my blog. When I change jobs, I edit `cv.typ`, push, and the new PDF is live. No "download from Google Docs, re-export, upload to website" dance. No version mismatch between what's on my site and what I emailed to someone last month.

The rough edges

Typst is young. Things that could be better:

- **cmarker isn't perfect.** Some markdown edge cases (nested lists with code blocks, complex tables) don't render exactly right. I haven't hit a showstopper, but I occasionally adjust the markdown to make cmarker happy.

- **No Zola shortcodes in PDFs.** The preprocessing strips shortcodes, which means figures and interactive elements don't appear in the PDF. I add a "view on site" note for anything stripped.
- **Font fallback is limited.** Typst's font fallback is less sophisticated than a browser's. I ship my fonts explicitly via `--font-path` to avoid surprises.
- **No incremental compilation.** Every deploy recompiles every PDF. For a handful of posts this is fine (sub-second each). At 100 posts I'd want to add change detection.

None of these are blockers. The output quality is excellent and the developer experience is leagues ahead of LaTeX.

The toolchain

Role	Tool
Typesetting engine	Typst
Markdown → Typst	cmarker
LaTeX math → Typst	MiTeX
Body font	Literata (serif)
Heading font	Inter (sans-serif)
Code font	JetBrains Mono

The blog post template, CV source, build scripts, and font files are all in the site repo. If you're using Zola (or any static site generator) and want auto-generated PDFs, the `generate-pdf.sh` script is the piece to look at – it's generic enough to adapt.

This post is part of a series on the infrastructure behind this blog. See also: Site overview, Self-hosted newsletter, Citations, Images.